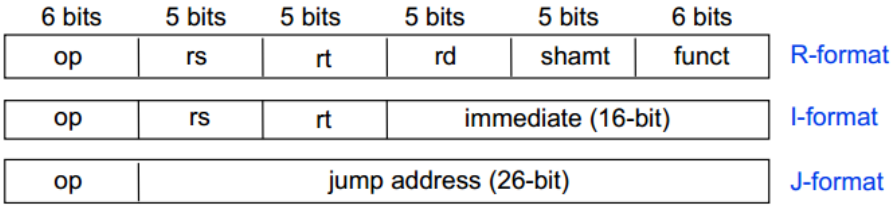
Tutorial 6: MIPS assembly language

Q1. Define and elaborate MIPS processor three main instruction formats. Using suitable diagram, briefly elaborate the three basic MIPS processor's instruction format with example instruction.

Instruction format specify the layout of the instruction bits in fields. Three basic MIPS processor instruction format are shown in figure below. The set of instructions can be represented in the following forms

- I-format :
  - Instructions with immediates: e.g. addi, ori, andi, etc
  - Data transfer: lw and sw (since the offset counts as an immediate)
    e.g. lw $s1, 8($0), sw $t1, 4($s4), lb $t2, 1($s1)
  - Program control: branches (beq and bne), but not the shift instructions
- J-format : for j and jal
- R-format : for all other instructions: e.g. add, sub, sll, and, or, etc



Q2. [Instruction Class : Logical instruction]
Perform multiplication function below by using MIPS logical instruction. Hence state the implication as compare to arithmetic instruction.
`f *= 8;` (in C code)

Compile into MIPS assembly code :

```
$s0 = f
sll  $s0,  $s0, 3        # $s0  =  $s0  <<3 bits, shift left 3 bits and fills emptied
                         #bits by zero extending (in MIPS code)
```

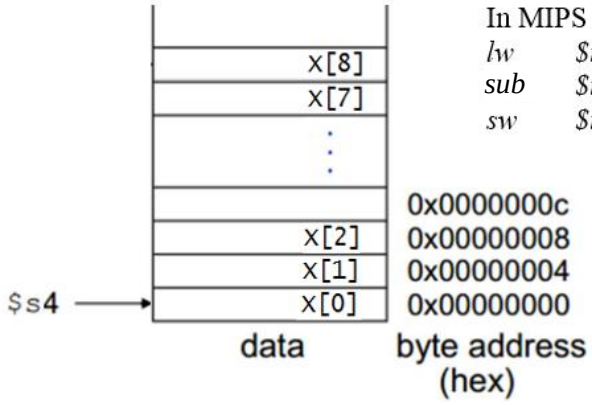
If using arithmetic instruction:

```
addi    $t2, $0, 8       #$t2 = 8
multu   $s0,   $t2       #multiple $s0 with $t2, save into Hi,Lo
mflo    $t0              #mov lower bytes in Lo into $t0  OR
mfhi    $t1              #mov higher bytes in Hi into $t1
```

* Multiplication instruction will yield slower respond as compare to logical instruction

Q3. [Instruction Class : Memory access & Data transfer instruction]
Assuming variable `y` stored in `$s3` and the base address of array X is in `$s4`, compile the C code of `X[8] = X[2] - y` to MIPS assembly code.

Assuming variable y stored in $s3 and the base address of array X is in $s4
X[8] = X[2] - y
$s3 : y
$s4 : base address of array X



Q4. [Program Control instruction : Conditional instruction]
Compile the C statement below into MIPS code using two different conditional instructions as in (a) and (b).

```
If ( a == b ) f = x – y;
Else f = x + y
```

(a) Using Conditional Equality instruction
(b) Using Conditional Inequality instruction

(a) Using Conditional Equality instruction, variable to registers mapping as below
f: $s0, x: $s1, y: $s2, a:$s3, b: $s4

```
        beq   $s3, $s4, True      # if(a==b) goto True
        add   $s0, $s1, $s2       # else f = x + y
        j       Exit              # goto Exit
True :  sub    $s0, $s1, $s2      # f = x – y (true)
Exit:
```

(b) Using Conditional Inequality instruction, variable to registers mapping as below
f: $s0, x: $s1, y: $s2, a:$s3, b: $s4

```
        bne   $s3, $s4, Else      # if(a!=b) goto Else
        sub   $s0, $s1, $s2       #  f = x - y
        j       Exit              # goto Exit
Else :  add   $s0, $s1, $s2       # f = x + y (skipped if a==b)
Exit:
```

Q5. Consider the MIPS assembly code below.

```
sub  $s0, $t1, $t0
mult $s0, $t2
mflo $s1
addi $s2, $s1, 0x5C
```

Assume that the registers `$t0`, `$t1` and `$t2` hold the decimal values of 20, 45 and 6, respectively. State the values of `$s0`, `$s1` and `$s2`.

`$s0` = 45 – 20 = 25
`$s1` = 25 × 6 = 150
`$s2` = 150 + 0x5C = 242

Q6. Give three examples from the MIPS architecture of each of the architecture design principles:

(1) simplicity favors regularity;
(2) make the common case fast;
(3) smaller is faster; and
(4) good design demands good compromises.

Explain how each of your examples exhibits the design principle.

(1) Simplicity favors regularity (any 3 points):
- Each instruction has a 6-bit opcode.
- All (R-type) instructions have 3 operands.
- MIPS has only 3 instruction formats (R-Type, I-Type, J-Type).
- Each instruction is the same size, making decoding hardware simple.
- Data is in words => make instructions in words too.

(2) Make the common case fast.
- Registers make the access to most recently accessed variables fast.
- Create only regularly used instructions as much as possible.
- Allow instructions to contain immediate operands

(3) Smaller is faster (any 3 points):
- The register file has limited number of registers i.e. 32. Small address decoder. And can have multiple ports to support arithmetic / logic operations in one cycle.
- The ISA includes only a small instruction set – smaller hardware.
- Limited number of addressing modes

(4) Good design demands good compromises.
- To have fixed-length instructions but different formats. MIPS uses three instruction formats (instead of just one).
- Also supports main memory accesses (not just register).
- Provides pseudocode to the user and compiler for commonly used operations.